



Vidyavardhini's College of Engineering & Technology

Department of Computer Engineering

Experiment No. 10
Develop test cases for White Box Testing for a given code
Date of Performance: 30/09/23
Date of Submission: 09/10/23



Vidyavardhini's College of Engineering & Technology

Department of Computer Engineering

Aim: To develop test cases for White Box Testing for a given code.

Objective: To analyse the code of Hotel Management System and develop test cases for White Box Testing.

Theory:

White Box Testing is software testing technique in which internal structure, design and coding of software are tested to verify flow of input-output and to improve design, usability and security. In white box testing, code is visible to testers so it is also called Clear box testing, Open box testing, Transparent box testing, Code-based testing and Glass box testing.

It is one of two parts of the Box Testing approach to software testing. Its counterpart, Blackbox testing, involves testing from an external or end-user type perspective. On the other hand, White box testing in software engineering is based on the inner workings of an application and revolves around internal testing.

The term “Whitebox” was used because of the see-through box concept. The clear box or Whitebox name symbolizes the ability to see through the software’s outer shell (or “box”) into its inner workings. Likewise, the “black box” in “Black Box Testing” symbolizes not being able to see the inner workings of the software so that only the end-user experience can be tested.

Steps:

STEP 1) UNDERSTAND THE SOURCE CODE

The first thing a tester will often do is learn and understand the source code of the application. Since white box testing involves the testing of the inner workings of an application, the tester must be very knowledgeable in the programming languages used in the applications they are testing. Also, the testing person must be highly aware of secure coding practices. Security is often one of the primary objectives of testing software. The tester should be able to find security issues and prevent attacks from hackers and naive users who might inject malicious code into the application either knowingly or unknowingly.

Step 2) CREATE TEST CASES AND EXECUTE

The second basic step to white box testing involves testing the application’s source code for proper flow and structure. One way is by writing more code to test the application’s source code. The tester will develop little tests for each process or series of processes in the application. This method requires that the tester must have intimate knowledge of the code and is often done by the developer. Other methods include Manual Testing, trial, and error testing and the use of testing tools as we will explain further on in this article.



Types of White Box Testing

White box testing encompasses several testing types used to evaluate the usability of an application, block of code or specific software package. There are listed below —

- **Unit Testing:** It is often the first type of testing done on an application. Unit Testing is performed on each unit or block of code as it is developed. Unit Testing is essentially done by the programmer. As a software developer, you develop a few lines of code, a single function or an object and test it to make sure it works before continuing. Unit Testing helps identify a majority of bugs, early in the software development lifecycle. Bugs identified in this stage are cheaper and easy to fix.
- **Testing for Memory Leaks:** Memory leaks are leading causes of slower running applications. A QA specialist who is experienced at detecting memory leaks is essential in cases where you have a slow running software application.

Advantages of White Box Testing

- Code optimization by finding hidden errors.
- White box tests cases can be easily automated.
- Testing is more thorough as all code paths are usually covered.
- Testing can start early in SDLC even if GUI is not available.

Disadvantages of White Box Testing

- White box testing can be quite complex and expensive.
- Developers who usually execute white box test cases detest it. The white box testing by developers is not detailed can lead to production errors.
- White box testing requires professional resources, with a detailed understanding of programming and implementation.
- White-box testing is time-consuming, bigger programming applications take the time to test fully.

Solution:

Code:

```
import unittest

def find_max(a, b, c):
    if a >= b:
        if a >= c:
            return a
        else:
            return c
    else:
        if b >= c:
            return b
```



Vidyavardhini's College of Engineering & Technology

Department of Computer Engineering

else:

return c

```
class TestFindMaxFunction(unittest.TestCase):
```

```
    def test_a_is_max(self):
```

```
        self.assertEqual(find_max(5, 3, 2), 5)
```

```
    def test_b_is_max(self):
```

```
        self.assertEqual(find_max(3, 7, 2), 7)
```

```
    def test_c_is_max(self):
```

```
        self.assertEqual(find_max(3, 5, 9), 9)
```

```
    def test_a_equals_b(self):
```

```
        self.assertEqual(find_max(4, 4, 2), 4)
```

```
    def test_b_equals_c(self):
```

```
        self.assertEqual(find_max(1, 3, 3), 3)
```

```
    def test_a_equals_c(self):
```

```
        self.assertEqual(find_max(6, 2, 6), 6)
```

```
    def test_all_equal(self):
```

```
        self.assertEqual(find_max(2, 2, 2), 2)
```

```
    def test_negative_numbers(self):
```

```
        self.assertEqual(find_max(-2, -5, -1), -1)
```

```
    def test_float_numbers(self):
```

```
        self.assertEqual(find_max(2.5, 1.8, 3.2), 3.2)
```

```
    def test_boundary_values(self):
```

```
        # Test with the minimum and maximum values for integers
```

```
        self.assertEqual(find_max(-2147483648, 0, 2147483647), 2147483647)
```

```
if __name__ == '__main__':
```

```
    unittest.main()
```

Output:

Ran 10 tests in 0.000s

OK



Code:

```
import unittest

def find_max(a, b, c):
    if a >= b:
        if a >= c:
            return a
        else:
            return c
    else:
        if b >= c:
            return b
        else:
            return c

class TestFindMaxFunction(unittest.TestCase):

    def test_a_is_max(self):
        self.assertEqual(find_max(5, 3, 2), 2)

    def test_b_is_max(self):
        self.assertEqual(find_max(3, 7, 2), 2)

    def test_c_is_max(self):
        self.assertEqual(find_max(3, 5, 9), 9)

    def test_a_equals_b(self):
        self.assertEqual(find_max(4, 4, 2), 4)

    def test_b_equals_c(self):
        self.assertEqual(find_max(1, 3, 3), 3)

    def test_a_equals_c(self):
        self.assertEqual(find_max(6, 2, 6), 6)

    def test_all_equal(self):
        self.assertEqual(find_max(2, 2, 2), 2)

    def test_negative_numbers(self):
        self.assertEqual(find_max(-2, -5, -1), -1)

    def test_float_numbers(self):
        self.assertEqual(find_max(2.5, 1.8, 3.2), 3.2)

    def test_boundary_values(self):
        # Test with the minimum and maximum values for integers
        self.assertEqual(find_max(-2147483648, 0, 2147483647), 2147483647)
```



Vidyavardhini's College of Engineering & Technology

Department of Computer Engineering

```
if __name__ == '__main__':  
    unittest.main()
```

Output:

..F..F..ERROR!

..

=====

FAIL: test_a_is_max (__main__.TestFindMaxFunction.test_a_is_max)

Traceback (most recent call last):

File "<string>", line 20, in test_a_is_max

AssertionError: 5 != 2

=====

FAIL: test_b_is_max (__main__.TestFindMaxFunction.test_b_is_max)

Traceback (most recent call last):ERROR!

File "<string>", line 23, in test_b_is_max

AssertionError: 7 != 2

Ran 10 tests in 0.001s

FAILED (failures=2)

Traceback (most recent call last):

File "<string>", line 51, in <module>

File "/usr/local/lib/python3.11/unittest/main.py", line 102, in __init__

self.runTests()

File "/usr/local/lib/python3.11/unittest/main.py", line 276, in runTests

sys.exit(not self.result.wasSuccessful())

SystemExit: True

>



Vidyavardhini's College of Engineering & Technology

Department of Computer Engineering

Conclusion: